Capstone Project Report

Iteration #04

DevInsight: AI-Powered Developer Productivity Prediction System

Team Members

Lakshmi Vadhanie	002335829
Vaishnavi Bhutada	002313263
Pranav Raghavendra Rao	002470519

DS 5500 - Data Science Capstone
Northeastern University

Contents

1 Introduction **2**

2 Purpose of the Methodology **2**

3 Problem Statement **4**
3.1 Defining the Problem 4
3.2 Significance of the Problem 4

4 Data Collection and Preparation **5**
4.1 Data Sources 5
4.2 Data Description 5
4.3 Preprocessing Steps 6

5 Selection of Machine Learning and LLM Models **6**
5.1 Model Consideration 6
5.2 Final Model Selection 7

6 Model Development and Training **8**
6.1 Architecture and Configuration 8
6.2 Training Process 8
6.3 Hyperparameter Tuning 9

7 Evaluation and Comparison **9**

1 Introduction

This project presents an end-to-end system for analyzing developer activity and predicting productivity patterns using a combination of data engineering, machine learning, and natural language processing techniques. The core idea is to leverage publicly available GitHub activity data to understand how different developer behaviors, such as pull request activity, code reviews, and issue interactions, influence future code contributions.

The system follows a layered architecture, beginning with data ingestion from the GitHub Archive dataset into BigQuery, where relevant repositories and event types are filtered and transformed into a structured format. This structured data is further processed into curated analytical tables that capture developer-level, pull request-level, and team-level metrics. These tables serve as the foundation for feature engineering and modeling.

On top of this data pipeline, a forecasting task is defined where the goal is to predict a developer's next-day commit activity based on their recent engagement patterns. Multiple models, including baseline approaches and a Long Short-Term Memory (LSTM) network, are implemented and compared to evaluate the importance of temporal dependencies in developer behavior.

In addition to structured data analysis, a parallel text processing pipeline extracts commit messages to enable future integration of transformer-based models for generating human-readable insights. The final system is supported by a FastAPI backend, enabling the deployment of predictions and insights through application interfaces such as dashboards or conversational assistants.

2 Purpose of the Methodology

The methodology adopted in this project is designed to effectively capture and model developer productivity patterns from large-scale, real-world activity data. A layered data engineering approach is used to transform raw GitHub event data into structured and meaningful representations, enabling robust analysis and modeling.

The use of curated analytical tables allows for the extraction of developer-level, pull request-level, and team-level metrics, which serve as reliable indicators of developer engagement. This structured representation is critical for reducing noise in raw event data and improving the quality of features used in downstream modeling.

A key objective of this methodology is to understand whether developer engagement signals can be used to predict future productivity. To achieve this, a forecasting framework is developed where the next-day commit activity is predicted based on historical behavioral patterns.

Multiple models are evaluated, including a mean baseline, linear regression, and a Long Short-Term Memory (LSTM) network. The comparison is essential to determine whether simple statistical relationships are sufficient or whether temporal dependencies play a significant role in modeling developer behavior.

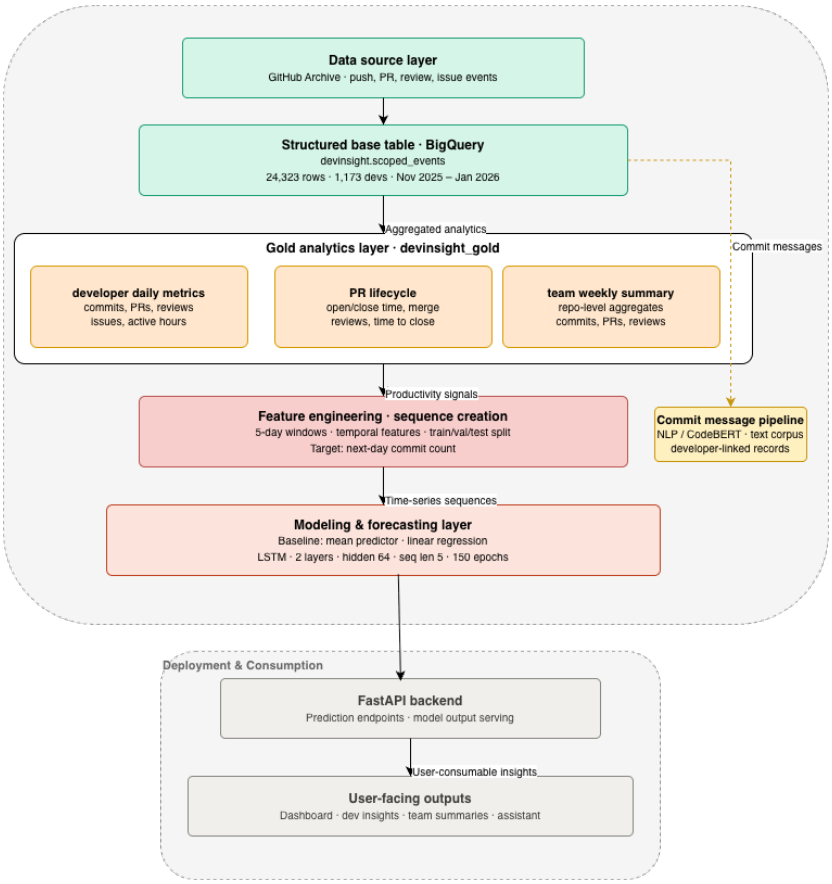

Figure 1: End-to-end architecture of the DevInsight system, illustrating data ingestion, transformation, modeling, and deployment components.

The inclusion of the LSTM model allows the system to capture sequential patterns in developer activity, providing deeper insights into how past actions influence future output.

Additionally, a parallel text processing pipeline is introduced to extract commit messages, enabling future integration of natural language processing techniques for generating interpretable, human-readable insights. Together, these methodological choices provide a comprehensive framework for both predictive modeling and actionable insight generation.

3 Problem Statement

3.1 Defining the Problem

The primary objective of this project is to model and predict developer productivity based on historical activity patterns. Specifically, the problem is formulated as a supervised machine learning regression task, where the goal is to predict a developer's next-day commit count using their recent engagement signals.

Developer productivity in this context is represented through observable behavioral metrics such as pull request activity, code reviews, and issue interactions. Rather than relying directly on past commit counts as input, the model is designed to use engagement-based features to avoid trivial or leakage-prone predictions. This ensures that the model learns meaningful relationships between developer behavior and future output.

To capture temporal dependencies in developer activity, the problem is further structured as a time-series forecasting task. Sequential patterns are constructed using rolling windows of historical data, allowing the model to learn how past actions influence future productivity over time.

In addition to structured behavioral data, the project also incorporates a parallel natural language processing component. Commit messages are extracted and prepared as a text corpus, enabling future exploration of transformer-based models for generating insights and enhancing interpretability.

3.2 Significance of the Problem

Understanding and predicting developer productivity has significant implications in both academic research and real-world software engineering environments. From an industry perspective, engineering teams often seek to optimize development workflows, improve collaboration, and identify bottlenecks in the software delivery process. A data-driven approach to modeling developer behavior can provide valuable insights into how different activities contribute to overall productivity.

From a research standpoint, this problem lies at the intersection of behavioral analytics, time-series modeling, and machine learning. It extends beyond simple performance measurement by attempting to uncover predictive relationships between engagement patterns and future output. The inclusion of temporal modeling further highlights the importance of sequential dependencies in human-driven processes.

Additionally, the integration of textual data through commit messages opens the possibility of combining structured and unstructured data for richer analysis. This can support the generation of human-readable insights, making the system more interpretable and actionable for end users.

Overall, solving this problem contributes to the development of intelligent systems that can assist in understanding developer behavior, improving team ef-

ficiency, and enabling data-driven decision-making in software engineering practices.

4 Data Collection and Preparation

4.1 Data Sources

The primary data source for this project is the GitHub Archive dataset, accessed through Google BigQuery. This public dataset contains a comprehensive record of GitHub events, including commits, pull requests, reviews, and issue-related activities. The use of this dataset enables the analysis of real-world developer behavior at scale.

To ensure relevance and quality, the dataset is filtered to include activity from selected large-scale open-source repositories, such as Apache Spark, Kubernetes, Apache Airflow, TensorFlow, and Prometheus. Additionally, only specific event types that are directly related to developer activity are considered, including PushEvent, PullRequestEvent, PullRequestReviewEvent, and IssuesEvent.

The data ingestion and filtering process is implemented using a Python-based pipeline, which interacts with BigQuery to create a structured dataset. The resulting base table, `scoped_events`, serves as the foundation for all subsequent transformations and modeling tasks.

4.2 Data Description

The curated dataset consists of 24,323 event records spanning a three-month period from November 2025 to January 2026. These records represent activity from 1,173 unique developers across the selected repositories.

Each record captures key attributes of developer activity, including the event timestamp, repository name, developer identifier, and event-specific metadata extracted from JSON payloads. These attributes enable the construction of meaningful behavioral features such as commit counts, pull request actions, review activity, and issue interactions.

The data exhibits characteristics typical of real-world developer activity, including sparsity and variability. In particular, commit activity is highly uneven, with many days containing zero commits and occasional spikes in activity. This imbalance introduces challenges for modeling and requires careful feature design to ensure meaningful predictions.

In addition to structured event data, commit messages are extracted from PushEvent payloads to create a text corpus. This corpus is intended for use in natural language processing tasks, enabling future integration of transformer-based models for generating insights.

4.3 Preprocessing Steps

The data preparation process involves multiple stages of transformation to convert raw event data into a structured and modeling-ready format. Initially, raw GitHub events are filtered based on repository, event type, and time range. Relevant fields are extracted from nested JSON payloads to create a structured representation of developer activity.

Following this, a set of curated analytical tables is created in a dedicated gold dataset. These tables include developer-level daily metrics, pull request lifecycle metrics, and team-level weekly summaries. The gold tables serve as a clean and aggregated representation of developer behavior, reducing noise and improving feature quality.

For the machine learning task, the data is further processed to construct a time-series dataset. Developer activity is aggregated at the daily level, and temporal features such as day of the week, weekend indicators, and week number are added. Sequential input windows of five days are created to capture temporal dependencies in developer behavior.

To ensure robust modeling, features that could introduce data leakage, such as direct commit counts and active hours, are excluded from the input variables. The target variable, next-day commit count, is log-transformed to reduce the impact of extreme values and improve model stability.

Finally, the dataset is split into training and testing sets using a chronological approach, ensuring that future data is not used during training. Feature scaling is applied using statistics derived from the training set, maintaining consistency and preventing information leakage.

For the text data pipeline, commit messages are extracted and stored in a structured format. Basic preprocessing is performed, including filtering null values and associating each message with its corresponding developer and repository. This prepared text corpus enables future experimentation with embedding models and transformer-based architectures.

5 Selection of Machine Learning and LLM Models

5.1 Model Consideration

To address the problem of predicting developer productivity, multiple modeling approaches were considered, ranging from simple statistical baselines to more advanced machine learning models. The goal was to evaluate whether developer engagement signals exhibit simple linear relationships or more complex temporal dependencies.

A mean baseline model was first considered as a reference point. This model predicts future commit activity using the average of past observations and serves as a minimal benchmark for evaluating model performance.

Next, a linear regression model was implemented to capture linear relationships between engagement features and future commit counts. This model provides interpretability and allows for the analysis of feature importance, offering insights into how different developer activities influence productivity.

However, developer behavior is inherently sequential and time-dependent. Activities such as pull requests, reviews, and issue handling often occur in patterns over time. To capture these temporal dependencies, a Long Short-Term Memory (LSTM) network was selected. LSTM models are specifically designed for sequential data and are capable of learning long-term dependencies, making them well-suited for time-series forecasting tasks.

In addition to structured data modeling, a natural language processing component was explored by extracting commit messages. These textual data provide contextual information about developer activity and can be leveraged using transformer-based models such as CodeBERT or GPT in future extensions of the system. While this component is not directly integrated into the predictive model in the current implementation, it lays the foundation for generating interpretable and human-readable insights.

To address the problem of developer productivity analysis, a diverse set of models was considered to capture different aspects of developer behavior. Rather than relying on a single model, the system incorporates multiple modeling approaches tailored to specific analytical tasks.

Behavioral Clustering Models: K-Means and DBSCAN were used to group developers based on engagement patterns such as commits, pull requests, reviews, and active hours. K-Means identifies structured clusters representing developer archetypes, while DBSCAN detects outliers and irregular behavioral patterns.

Isolation Forest and Variational Autoencoder were used to detect abnormal patterns in developer and team activity.

Forecasting Models: Multiple models were considered for predicting developer productivity, including Mean baseline, Linear Regression, LSTM, XGBoost, and Transformer-based models. Linear Regression provides interpretability, LSTM captures temporal dependencies, XGBoost models non-linear feature interactions, and Transformer models explore long-range dependencies at the team level.

Optimization Models: A contextual bandit model with Thompson Sampling was implemented to optimize sprint strategies by balancing exploration and exploitation of different development configurations.

5.2 Final Model Selection

The final model selection was based on empirical evaluation and the need to balance interpretability and predictive performance. Three models were compared: the mean baseline, linear regression, and the LSTM model.

The mean baseline establishes a lower bound for performance and helps determine whether the predictive task is meaningful. The linear regression model serves as a simple yet interpretable approach, enabling the identification of

relationships between engagement features and future commits. However, it assumes independence between observations and does not capture temporal dynamics.

The LSTM model was selected as the final model due to its ability to model sequential dependencies in developer activity. By using rolling windows of past engagement signals, the LSTM captures patterns that evolve over time, leading to improved predictive performance compared to non-sequential models.

Model performance was evaluated using standard regression metrics, including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). The comparison across models demonstrated that incorporating temporal information significantly improves prediction accuracy, justifying the use of LSTM for this task.

The inclusion of multiple models in the evaluation process ensures that the chosen approach is not only effective but also well-justified, providing both empirical evidence and theoretical reasoning for the final selection.

6 Model Development and Training

6.1 Architecture and Configuration

The predictive modeling task is formulated as a time-series regression problem, where the objective is to predict a developer's next-day commit count based on recent engagement activity. To capture temporal dependencies, a Long Short-Term Memory (LSTM) network is employed as the primary model.

The input to the model consists of sequential windows of five consecutive days of developer activity. Each time step includes multiple behavioral and temporal features, such as pull request activity, review counts, issue interactions, and calendar-based indicators. This sequential representation allows the model to learn how patterns evolve over time.

The LSTM architecture consists of two stacked LSTM layers with a hidden dimension of 64 units. A dropout layer with a rate of 0.1 is applied to reduce overfitting and improve generalization. The final output layer is a fully connected layer that produces a single continuous value representing the predicted next-day commit count.

To ensure meaningful learning, features that could introduce data leakage, such as direct commit counts and active hours, are excluded from the input. This design forces the model to rely on indirect engagement signals rather than trivial predictors.

6.2 Training Process

The dataset is first aggregated at the developer-day level and then transformed into sequential samples using rolling windows of five days. Each sequence is paired with a target value corresponding to the commit count on the following day.

A chronological data split is used to divide the dataset into training and testing sets. This approach ensures that future information is not used during training, preserving the integrity of the time-series forecasting setup.

Feature scaling is applied to the input variables using statistics computed from the training set. This normalization step improves model convergence and stability during training.

The model is trained using the Mean Squared Error (MSE) loss function and optimized using the Adam optimizer. Training is conducted for 150 epochs, with a learning rate scheduler applied to adjust the learning rate dynamically. Gradient clipping is also used to prevent exploding gradients and ensure stable training.

To monitor model performance, both training and validation losses are tracked across epochs. The observed training behavior indicates stable convergence with minimal overfitting, as the validation loss stabilizes early during training.

6.3 Hyperparameter Tuning

The model configuration is selected based on empirical experimentation and performance evaluation. Key hyperparameters include the sequence length (five days), hidden layer size (64 units), dropout rate (0.1), and number of training epochs (150).

Rather than using exhaustive search techniques such as grid search or Bayesian optimization, a manual tuning approach is adopted. Different configurations are tested iteratively, and the final parameters are chosen based on their ability to balance model performance and training stability.

In addition to the LSTM model, baseline models such as mean prediction and linear regression are also implemented and evaluated. This comparison provides a reference for assessing the effectiveness of the chosen architecture and ensures that the improvements achieved by the LSTM model are meaningful.

Overall, the model development process emphasizes both predictive performance and methodological rigor, ensuring that the final model is well-suited for capturing temporal patterns in developer activity.

7 Evaluation and Comparison

To evaluate the effectiveness of the proposed models, multiple performance metrics are used, including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). These metrics provide a comprehensive assessment of prediction accuracy, error magnitude, and the model's ability to explain variance in the data.

Three models are compared: a mean baseline model, a linear regression model, and the LSTM model. The mean baseline serves as a reference point, representing a naive approach that does not consider any input features. As expected, this model performs poorly, with near-zero or negative R^2 values, indicating that it fails to capture meaningful patterns in the data.

The linear regression model demonstrates a significant improvement over the baseline, achieving an R^2 score of approximately 0.44. This indicates that developer engagement features do have a measurable relationship with future commit activity. Additionally, the linear model provides interpretability through feature importance analysis, allowing insights into which behaviors contribute most to productivity.

The LSTM model further improves performance, achieving an R^2 score of approximately 0.66. This improvement highlights the importance of temporal dependencies in developer behavior. By modeling sequential patterns, the LSTM captures relationships that cannot be represented by static models such as linear regression.

Analysis of the prediction results reveals that developer activity is highly sparse and variable, with many days of low activity and occasional spikes in commit counts. While the LSTM model is able to capture overall trends more effectively than the linear model, it still faces challenges in predicting extreme spikes. This is expected due to the stochastic nature of human-driven behavior.

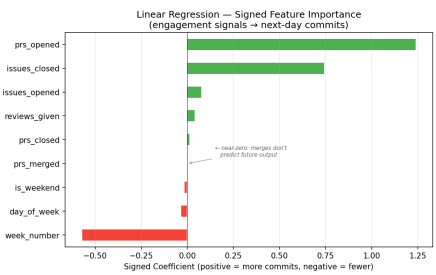

Figure 2: Model performance comparison including baseline, linear regression, and LSTM.

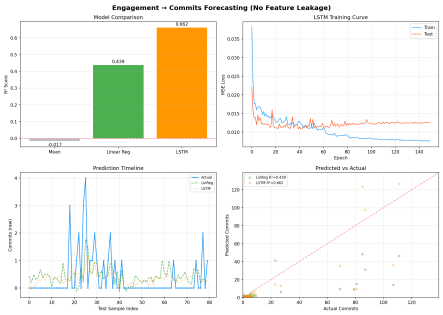

Figure 3: Feature importance showing contribution of engagement signals.

As shown in Figure 2, the LSTM model captures temporal patterns in developer activity more effectively than baseline models, resulting in improved predictive performance.

Figure 3 highlights the importance of engagement features such as pull request activity and reviews in influencing future commit behavior.

Visualization of model performance further supports these findings. Training and validation loss curves indicate stable convergence, with minimal overfitting. Prediction plots show that the LSTM model follows actual activity trends more closely than the baseline models, while scatter plots of predicted versus actual values demonstrate a wider and more accurate distribution of predictions.

In addition to predictive performance, feature importance analysis from the linear regression model provides valuable interpretability. Engagement features such as pull request activity and review participation show meaningful contributions to future commit behavior, reinforcing the importance of collaborative development activities.

Overall, the evaluation results demonstrate that incorporating temporal modeling significantly enhances predictive performance. The comparison across models validates the choice of LSTM as the final model and highlights the value of combining structured behavioral data with sequential learning techniques.

In conclusion, this project demonstrates the effectiveness of combining large-scale data engineering, feature-driven modeling, and sequential learning techniques to understand and predict developer productivity. The results highlight the importance of temporal dependencies in developer behavior, with the LSTM model significantly outperforming traditional baseline approaches.

Overall, the proposed system provides a scalable and extensible framework that can support data-driven decision-making in software engineering environments, while also enabling future enhancements through the integration of natural language processing and real-time application interfaces.